# MEAM 5200 Final Project Report

Team 9: Simulation Quality Control

Sohan Lele Boyan Zhang Hongbo Nie Jiayuan Chen

May 12, 2026

## 1 Introduction

The MEAM 5200 final project is a class-wide robotic system for serving custom-ordered trail mix at the ROBO MS thesis reception. The full system uses four Franka Panda arms arranged as two assembly lines. In each line, the service robot picks up a cup, dispenses the cereal base, mixes the final product, and serves it to the customer. The toppings robot adds nuts and candy. The two robots do not hand cups directly to each other. Instead, they transfer cups using a turntable, which keeps the robot workspaces separated and makes the system safer.

Because the project was divided across 20 teams, no single team could fully test its code in isolation. The frontend, scheduler, communications layer, simulator, perception modules, turntable, and robot action teams all depended on shared assumptions about messages, topics, object names, and expected robot behavior. Team 9's role was Simulation Quality Control. Our goal was not to write code that would run during the final hardware demo. Instead, we built development and integration tools that other teams could use before the complete system was available.

Our final deliverables consisted of two main pieces. First, we wrote simulation validation scripts that check whether important Gazebo components behave correctly. These tests cover deployment readiness, cup stability, dispenser lever behavior, turntable motion, gripper command behavior, and arm motion. Second, we wrote mock robot nodes that imitate the service and toppings robot teams. These mocks let the scheduler, communications, and frontend teams test their pipelines without waiting for the real robot controllers from Teams 11 through 19.

## 2 Project Scope

Team 9 was responsible for testing the simulation and providing mock robot behavior. We were not responsible for creating the Gazebo world, writing the scheduler, designing the ROS message definitions, implementing perception, or controlling the physical robots. Our work was meant to sit next to the main runtime system as a quality-control layer.

The project specification describes Team 9 as the team responsible for constructing test scenarios for the simulation, writing control code that checks simulator outputs, providing control inputs, and checking the resulting state. It also notes that, in the absence of functional robot code, this code could be used to test communications, the frontend, and the scheduler. We interpreted this as two connected goals:

1. Build small, direct tests that tell other teams whether the simulation substrate is currently usable.
2. Build mock robot stand-ins that allow upstream software teams to test message passing and sequencing before the real robot code is complete.

This scope shaped the rest of our design. We prioritized tools that were simple to run, easy to debug, and useful to other teams. We avoided building a large testing framework because that would have introduced more complexity than the project needed.

# 3 System Architecture and Interfaces

The complete trail mix system begins with a customer order from the frontend. The high-level scheduler chooses which assembly line should handle the order. The mid-level scheduler turns the order into robot-level actions. These commands are sent through Team 2's ROS communications layer. The robot action teams then execute the required physical tasks.

Team 9 connects to this architecture in two places. For simulation tests, we interact directly with Gazebo services and topics. For mock robot tests, we interact with Team 2's communication interface by subscribing to task commands and publishing robot status messages.

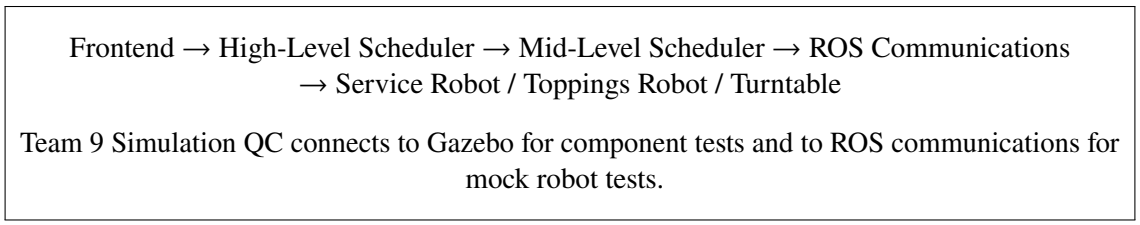


Figure 1: High-level relationship between Team 9's tools and the class-wide trail mix system.

Our code builds against three main dependencies:

**Team 2 communications.** Team 2 provided the ROS message definitions and the `TrailMixInterface` wrapper. Our mock robot nodes use this wrapper instead of hardcoding raw topic strings. This makes our code more robust if topic names change but the interface object is updated correctly.

**Team 8 simulation.** Team 8 provided the Gazebo world with Franka arms, cups, dispensers, and the turntable. Our Phase 1 tests query and command this world using Gazebo services such as `/gazebo/get_model_state`, `/gazebo/set_model_state`, and `/gazebo/apply_joint_effort`.

**Existing lab infrastructure.** The arm tests use the existing `ArmController` interface from the course labs. This allowed us to test whether the simulated Frankas could be commanded through the same style of interface used in previous assignments.

The development environment was Ubuntu 20.04, ROS Noetic, Python 3.8, and a `catkin_make_isolated` workspace rooted at `~/meam520_ws`.

# 4 Design Principles

Before writing the final test suite, we chose a few design principles to keep the project realistic and useful.

**Plug-and-play usage.** Other teams should be able to run a test with one command and immediately understand the output. This led us to standalone Python scripts instead of a custom package or large test framework.

**One question per test.** Each test answers one clear question. For example, `test_cup_stack.py` asks whether a cup stays stable in the simulation. `test_dispenser_lever.py` asks whether the dispenser lever moves and returns. Keeping tests narrow makes failures easier to interpret.

**Actionable failures.** A failing test should tell the user what failed and which component is likely responsible. This matters because our tests are meant to be used by other teams, not just by us.

**Simple code over clever code.** A short script that calls Gazebo directly is better than an abstract framework that only our team understands. Since the final project was short, maintainability and readability mattered more than maximizing abstraction.

**Clean handling of missing dependencies.** Team 8 delivered the simulation progressively, so some components were not always present. A missing component should not look the same as a broken component. For this reason, our tests use three exit codes:

$$0 = \text{PASS}, \quad 1 = \text{FAIL}, \quad 2 = \text{SKIP}.$$

A `SKIP` means the required simulation object is not deployed yet. This made the test suite useful even before the full simulation was complete.

# 5 Repository Structure

The repository is intentionally flat. It does not contain a full ROS package, `setup.py`, `package.xml`, or `CMakeLists.txt`. Our code is a set of standalone scripts that import from existing course and team packages.

The main files are:

- `README.md`: project overview, design principles, known gaps, and build instructions.
- `helpers.py`: shared constants and helper functions for Gazebo and `trail_mix` messages.
- `run_all.sh`: runs every `test_*.py` script and summarizes pass, fail, and skip counts.
- `service_mock.py`: mock service robot node.
- `toppings_mock.py`: mock toppings robot node.
- `test_deployment.py`: deployment readiness diagnostic.
- `test_cup_stack.py`: cup stability test.
- `test_dispenser_lever.py`: dispenser lever test.
- `test_turntable.py`: turntable motion and cup carry test.
- `test_gripper_grasp.py`: gripper open and close smoke test.
- `test_arms_move.py`: Franka arm motion test.
- `test_e2e_happy_path.py`: single-order mock end-to-end test.

This structure made the repository easy for other teams to use. There is no build step for Team 9's code itself. As long as the workspace is sourced and the required upstream packages are available, the scripts can be run directly.

# 6 Implementation

## 6.1 Shared Helper Layer

The helper file contains constants for model names and joint names, along with thin wrappers around Gazebo services. For example, instead of each test separately implementing model lookup or pose retrieval, the tests call shared helper functions such as `wait_for_sim()`, `get_pose()`, `set_pose()`, `list_gazebo_models()`, and `require_models()`.

The most important helper is `require_models()`. At the start of each simulation test, this function checks whether the needed Gazebo models are present. If they are missing, the test prints a `SKIP` message and exits with code 2. This prevents partially deployed simulation worlds from being misclassified as broken.

The helper file also includes message builders for the mock robot system. The `build_robot_status()` and `build_robot_command()` functions create well-formed messages with valid `order_id` values and timestamps. This was necessary because Team 2's validator rejects invalid message fields. Centralizing message construction reduced the chance that the mocks or tests would fail due to preventable formatting mistakes.

## 6.2 Phase 0: Deployment Readiness

The deployment readiness script, `test_deployment.py`, is a diagnostic rather than a strict pass-fail test. It queries the Gazebo world for expected models and checks whether controller managers are running for expected robot namespaces.

This script compares the current simulation against the project-level expectation of multiple Frankas, turntables, dispensers, and cups. However, it always exits with code 0. Its purpose is to tell the user what is deployed, not to mark the build as failed. This was useful because the simulation was not delivered all at once. When other tests skipped, `test_deployment.py` helped explain why.

## 6.3 Phase 1: Simulation Component Tests

Phase 1 contains the main Gazebo component tests. These tests directly check whether specific simulation objects behave in ways required by the larger project.

### 6.3.1 `test_cup_stack.py`

This test checks whether a cup remains stable in the simulation. It reads the pose of a known cup model, waits five seconds, then reads the pose again. The test passes if the cup's position drift is less than 1 cm.

This test is simple, but it catches important simulation issues. If the cup stack is unstable, many downstream robot tests become meaningless because the robot controllers would be planning around objects that move unpredictably before being touched.

### 6.3.2 `test_dispenser_lever.py`

This test checks whether a dispenser lever can be actuated and whether it returns to its resting position. The script reads the initial joint angle, applies effort to the lever joint, reads the peak deflection, waits for the spring behavior to return the lever, and then reads the final joint angle.

The test passes if the lever moves by a meaningful amount and returns close to its starting angle. This catches several possible simulation failures, including missing joints, broken effort application, missing spring behavior, or incorrect dispenser URDF settings.

#### 6.3.3 `test_turntable.py`

This test checks whether the turntable rotates and whether a cup placed on the turntable moves with it. The script teleports an existing cup onto the turntable, applies effort to the turntable joint, and then checks both the rotating link and the cup pose.

The important design decision here was to command the turntable through Gazebo joint effort rather than relying on a velocity controller. The velocity controller path was not reliable because the expected controller manager service was not consistently available. Applying joint effort directly tested the behavior we cared about: whether the turntable joint could rotate and whether contact/friction caused the cup to ride along.

#### 6.3.4 `test_gripper_grasp.py`

This test is a gripper API smoke test. It creates an `ArmController`, moves the arm to neutral, opens the gripper with `arm._gripper.move_joints(0.08)`, and closes it with `arm._gripper.move_joints(0.00)`. The test passes if both commands complete without throwing an exception.

Earlier versions of this test attempted to verify full cup grasp contact physics. We decided not to include that as a Team 9 test because it was brittle and overlapped with Team 16's service robot QC responsibilities. Contact physics in Gazebo made the result sensitive to spawn position, controller compliance, and timing. A better layered test is for Team 16 to verify that its real pick-cup behavior actually holds a cup.

#### 6.3.5 `test_arms_move.py`

This test verifies that both simulated Franka arms respond to commands through the existing course controller interface. It sends `ArmController(id=1)` and `ArmController(id=2)` to a neutral pose and then to a second target pose. It reads joint states afterward and checks that the arms reach the commanded configurations within tolerance.

This test catches issues where one robot namespace is misconfigured, one controller stack is missing, or one arm fails to respond even though Gazebo itself is running.

### 6.4 Phase 2: Mock Robot Nodes

The mock robot nodes were built to support integration testing for Teams 1, 2, 5, and 6. The service mock pretends to be the service robot teams, and the toppings mock pretends to be the toppings robot teams.

The service mock handles actions such as:

- `pick_cup`
- `dispense_cereal`
- `mix`
- `exchange`
- `serve`

The toppings mock handles actions such as:

- `dispense_nuts`
- `dispense_candy`
- `exchange`

Each mock subscribes to `task_cmd` through `TrailMixInterface`. When it receives a command for its robot group, it publishes a `RobotStatus` message with status `in_progress`, sleeps for a plausible action duration, and then publishes another `RobotStatus` message with status `done`.

The mock behavior is intentionally minimal. We did not add failure injection, random timing variation, retry handling, or complex dispatch tables because no consumer team had requested them yet. The goal was to provide just enough behavior for other teams to test their own pipelines.

### 6.5 Phase 3: End-to-End Happy Path

The file `test_e2e_happy_path.py` implements a single-order integration test through the mock robot layer. It starts both mock nodes using `subprocess.Popen`, waits until both mocks are subscribed to the task command publisher, then publishes the expected eight-step command sequence for one trail mix order:

$$\texttt{pick\_cup} \rightarrow \texttt{dispense\_cereal} \rightarrow \texttt{mix} \rightarrow \texttt{exchange}$$
$$\rightarrow \texttt{dispense\_nuts} \rightarrow \texttt{dispense\_candy} \rightarrow \texttt{exchange} \rightarrow \texttt{serve}.$$

The test subscribes to `robot_status` before launching the mocks so that it does not miss early responses. It then waits until every expected `(robot_group, action)` pair returns a `done` status. If all eight acknowledgments arrive, the test passes. If any are missing, it fails and reports which pair was not completed.

This test deliberately bypasses the real scheduler. Instead of testing whether Team 6 can convert an order into commands, it directly publishes the commands that Team 6 would be expected to produce. This isolates Team 2 communication behavior and Team 9 mock behavior from scheduler logic.

## 7 Evaluation

### 7.1 Evaluation Strategy

Our evaluation focuses on whether each test answers a useful integration question. Because Team 9 is an infrastructure team, the goal is not to optimize a robot trajectory or maximize physical performance. The goal is to determine whether the simulation and communication layers are usable enough for other teams.

Each test reports one of three outcomes:

| Exit Code | Meaning | Interpretation |
|---|---|---|
| 0 | PASS | The checked component behaves as expected. |
| 1 | FAIL | The component is deployed but behaving incorrectly. |
| 2 | SKIP | A required dependency is not deployed yet. |

Table 1: Exit-code convention used across the Team 9 test suite.

The full suite can be run using:

```
./run_all.sh
```

The runner executes every `test_*.py` script, captures the exit code, and prints a summary with the number of passed, failed, and skipped tests. This gives other teams a quick view of the current simulation state.

### 7.2 Current Test Coverage

| Test | Main Question | Primary Dependency |
|---|---|---|
| `test_deployment.py` | Which expected models and controllers are present? | Gazebo, controllers |
| `test_cup_stack.py` | Does a cup stay stable over time? | Team 8 sim |
| `test_dispenser_lever.py` | Does the dispenser lever move and return? | Team 8 sim |
| `test_turntable.py` | Does the turntable rotate and carry a cup? | Team 8 sim |
| `test_gripper_grasp.py` | Does the gripper open and close through the supported API? | Lab infra |
| `test_arms_move.py` | Do both Franka arms move to commanded poses? | Lab infra, Team 8 sim |
| `test_e2e_happy_path.py` | Do mock robots complete an eight-step order sequence? | Team 2 comms |

Table 2: Summary of Team 9 test coverage.

### 7.3 Results

All results below were produced against the Team 8 simulation as of its latest published state prior to the final demonstration on May 8, 2026. Consistent with standard integration practice, we treated demonstration day as the integration cutoff for our QC pipeline: upstream changes pushed during or after the demonstration — notably Team 8's commit 54024d1 ("turntable fix"), pushed on May 8 — were considered out of scope, as integrating them would have invalidated the snapshot against which Team 9 had verified the test suite.

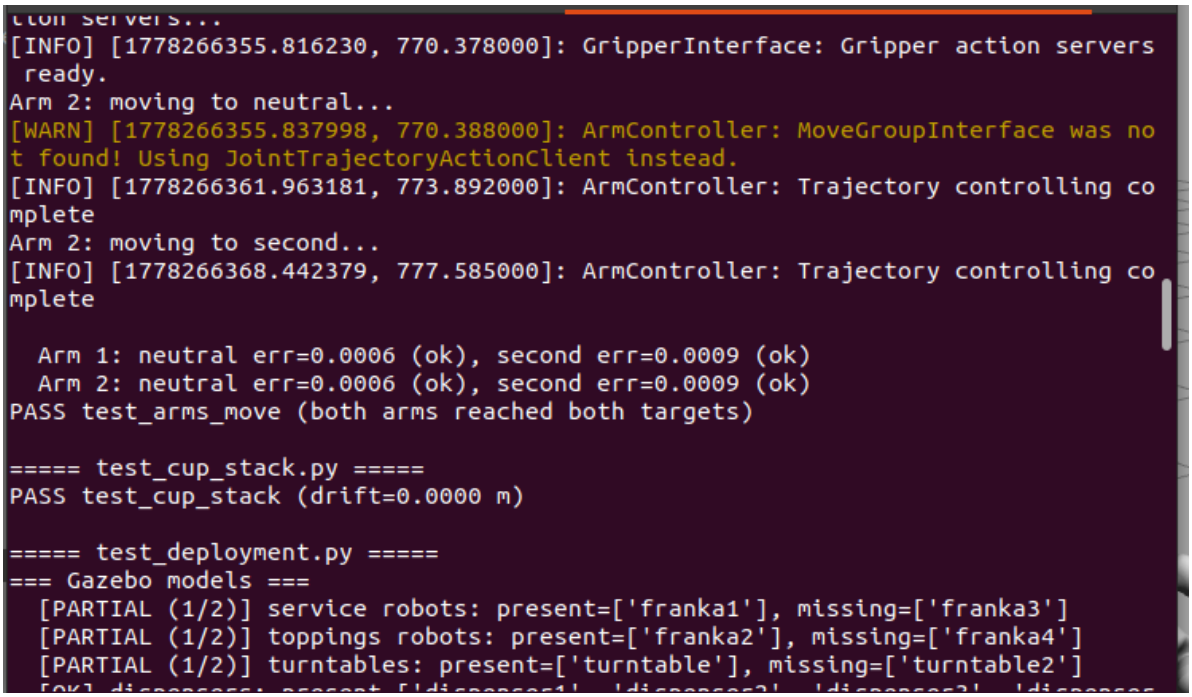


Figure 2: Output of `test_arms_move.py` and `test_cup_stack.py`.

```
4', 'dispenser5', 'dispenser6'], missing=[]
  [OK] cups: present=['cup_z_021', 'cup_z_022', 'cup_z_023', 'cup_z_024'], missi
ng=[]

=== Controller managers ===
  [OK] /franka1/controller_manager
  [SKIP] /franka3/controller_manager (model not in sim)
  [OK] /franka2/controller_manager
  [SKIP] /franka4/controller_manager (model not in sim)
  [MISSING] /turntable/controller_manager
  [SKIP] /turntable2/controller_manager (model not in sim)

(diagnostic only — exits 0 regardless. Tests that depend on
 missing components will SKIP, not FAIL.)

===== test_dispenser_lever.py =====
PASS test_dispenser_lever (neutral=0.005, peak=0.462, rest=0.028)

===== test_e2e_happy_path.py =====
[INFO] [1778266387.483914, 0.000000]: toppings_mock ready (actions: ['dispense_n
uts', 'dispense_candy', 'exchange'])
[INFO] [1778266387.561036, 788.433000]: service_mock ready (actions: ['pick_cup'
, 'dispense_cereal', 'mix', 'exchange', 'serve'])
```

Figure 3: Output of `test_deployment.py` and `test_dispenser_lever.py`.

```
===== test_e2e_happy_path.py =====
[INFO] [1778266387.483914, 0.000000]: toppings_mock ready (actions: ['dispense_n
uts', 'dispense_candy', 'exchange'])
[INFO] [1778266387.561036, 788.433000]: service_mock ready (actions: ['pick_cup'
, 'dispense_cereal', 'mix', 'exchange', 'serve'])
[INFO] [1778266387.641425, 788.475000]: service_mock: pick_cup order=42 (sleep 2
.0s)
[INFO] [1778266388.442992, 788.959000]: toppings_mock: dispense_nuts order=42 (s
leep 2.5s)
[INFO] [1778266391.026065, 790.476000]: service_mock: dispense_cereal order=42 (
sleep 3.0s)
[WARN] [1778266392.691154, 791.460000]: Topic /robot/status publish rate 0.40 Hz
 is below the minimum 0.5 Hz.
[INFO] [1778266392.692559, 791.461000]: toppings_mock: dispense_candy order=42 (
sleep 2.5s)
[INFO] [1778266396.154123, 793.477000]: service_mock: mix order=42 (sleep 2.0s)
[INFO] [1778266396.974962, 793.962000]: toppings_mock: exchange order=42 (sleep
2.0s)
[INFO] [1778266399.584818, 795.477000]: service_mock: exchange order=42 (sleep 2
.0s)
[INFO] [1778266403.017992, 797.477000]: service_mock: serve order=42 (sleep 2.0s
)
PASS test_e2e_happy_path (all 8 actions done)
```

Figure 4: Output of `test_e2e_happy_path.py`.

```
===== test_gripper_grasp.py =====
[INFO] [1778266408.910902, 800.879000]: Robot control running in Simulation Mode
!
[INFO] [1778266408.915955, 800.881000]: Waiting for collision behaviour services
...
[INFO] [1778266410.121530, 801.602000]: ArmController: Collision Service Not fou
nd. It will not be possible to change collision behaviour of robot!
[INFO] [1778266410.127447, 801.605000]: FrankaControllerManagerInterface: cannot
 detect controller name under param /franka1/controllers_config/impedance_contro
ller
[INFO] [1778266410.179867, 801.634000]: Moveit server does not seem to be runnin
g.
[INFO] [1778266410.182442, 801.634000]: ArmController: MoveGroup was not found!
This is okay if moveit service is not required!
[INFO] [1778266410.345772, 801.734000]: GripperInterface: Waiting for gripper ac
tion servers...
[INFO] [1778266410.542311, 801.841000]: GripperInterface: Gripper action servers
 ready.
[WARN] [1778266410.564518, 801.853000]: ArmController: MoveGroupInterface was no
t found! Using JointTrajectoryActionClient instead.
[INFO] [1778266417.052862, 805.639000]: ArmController: Trajectory controlling co
mplete
PASS test_gripper_grasp (open + close commands both completed)
```

Figure 5: Output of `test_gripper_grasp.py` and `test_turntable.py`.

```
[INFO] [1778266417.052862, 805.639000]: ArmController: Trajectory controlling co
mplete
PASS test_gripper_grasp (open + close commands both completed)

===== test_turntable.py =====
FAIL test_turntable (cup didn't follow: only 0.002 m < 0.02; check friction or c
up placement)

===== Summary =====
PASSED:  6
FAILED:  1
SKIPPED: 0
Failures:
  - test_turntable.py
```

Figure 6: Final summary block of `./run_all.sh`.

| Test | Result | Notes | Responsible Component |
|---|---|---|---|
| `test_deployment.py` | Diagnostic | 7 OK / 6 SKIP / 1 MISSING (turntable CM) | N/A |
| `test_cup_stack.py` | PASS | `test_cup_stack`: drift = 0.0000 m | Team 8 |
| `test_dispenser_lever.py` | PASS | `test_dispenser_lever`: neutral = 0.005, peak = 0.462, rest = 0.028 | Team 8 |
| `test_turntable.py` | FAIL | `test_turntable`: cup did not follow; only 0.002 m < 0.02 m. Check friction or cup placement. | Team 8 |
| `test_gripper_grasp.py` | PASS | `test_gripper_grasp`: open and close commands both completed | Team 8 / lab infra |
| `test_arms_move.py` | PASS | `test_arms_move`: both arms reached both targets | Team 8 / lab infra |
| `test_e2e_happy_path.py` | PASS | `test_e2e_happy_path`: all 8 actions done | Team 2 / Team 9 |

Table 3: Final test results.

Of the seven tests, six passed and one (**turntable.py**) failed; **test_deployment.py** served as a diagnostic. The failure is consistent with the diagnostic finding that although **turntable1** model was present in the simulation environment, its associated **turntable/controller_manager** ROS service was not advertised in the simulation environment, indicating an upstream an upstream controller-plumbing gap rather than a defect in the test harness. The remaining tests - covering cup stacking, dispenser dynamics, gripper actuation, dual arm motion, and the phase 3 end-to-end mock interface all completed within their tolerances.

# 8 Analysis

## 8.1 Why the Testing Scope Was Narrow

A major design decision was to keep each test narrow. This made the suite less flashy, but more useful. For example, the cup stability test does not attempt to evaluate every possible cup pose. It only checks whether a representative stacked cup stays stable. The dispenser test does not attempt to model cereal flow. It only checks whether the lever joint behaves correctly. This is enough to catch integration blockers without creating a separate research project.

This approach also made failures easier to assign. If `test_dispenser_lever.py` fails, the likely issue is in the dispenser model, spring behavior, or Gazebo effort application. If a larger all-in-one test failed, it would be much harder to identify the source.

## 8.2 Interface Gaps Discovered

While building the mock robot nodes, we found several interface gaps in the surrounding system.

1. **`RobotCommand` does not include an `assembly_line` field.** If both service robots subscribe to the same task command topic, the command does not clearly identify which assembly line should execute it. This matters once the system moves from one simulated line to two physical lines.

2. **Ingredient quantity types are inconsistent.** One part of the interface treats ingredient quantity as a boolean include/exclude value, while another treats command quantity as an integer. This creates ambiguity about whether the system supports multiple portion sizes.

3. **`CupState.location` appears incomplete.** There is no clear location state for the toppings robot holding the cup, which could make world-state tracking ambiguous.

4. **`target_id` values are underdocumented.** Actions such as `dispense_nuts` and `dispense_candy` imply target identifiers for nuts and candy, but the examples do not fully document those values.

5. **There is no unique per-command ID.** Without a unique command identifier, it is harder to distinguish a retry from a duplicate message or correlate timing across logs.

These gaps were not failures of Team 9's code, but they affected how robustly our mocks could behave. Surfacing these ambiguities was one of the useful outcomes of building against real shared interfaces early.

## 8.3 Gripper Sim-to-Real Issue

While working on the gripper test, we found a difference between simulation behavior and real robot behavior in the existing `ArmController` interface. In simulation, `arm.exec_gripper_cmd(width, force)` and `arm._gripper.grasp()` appeared to work. However, on the real robot, these paths

did not publish the expected goal when used with force-based grasping. The more reliable path was `arm._gripper.move_joints(width)`.

This mattered because a test using the wrong gripper function could pass in simulation while silently doing nothing on hardware. That would create a false sense of correctness. To avoid this, our final gripper smoke test uses `move_joints()` only.

### 8.4 Simulation-to-Reality Gap

The simulation was useful for development, but it could not fully predict hardware behavior. Contact-rich tasks such as grasping cups, manipulating dispenser levers, and carrying cups on the turntable are especially sensitive to friction, compliance, timing, and object geometry. For this reason, our tests should be interpreted as simulation readiness checks, not proof that the hardware demo will work.

This distinction is important. A passing simulation test means the simulated component is usable for software integration. It does not guarantee that the real robot will succeed. The final system still requires hardware testing by the robot action teams.

### 8.5 Incomplete Simulation Environment at the Deadline

Team 8's simulation environment was delivered progressively, and several components were still missing or non-functional by the report deadline. In the final integration window, only one of the two turntables spawned in the world, and the deployed turntable did not rotate because `/turntable/controller_manager` was not available and the `controller_spawner` stage of `turntable.launch` appeared to fail silently. Team 8 confirmed both issues but did not identify a root cause in time. These gaps blocked us from running a full two-line, two-turntable end-to-end test in Gazebo, and the situation is reflected directly in our results: `test_deployment.py` flags the missing controller manager, and `test_turntable.py` reports a failing cup-follow check.

We treat this outcome as an expected case in our design rather than an exceptional one. From the beginning, we assumed that partial delivery from Team 8 was the most likely development condition, which is why the `require_models()` helper, the three-state exit-code convention (`PASS` / `FAIL` / `SKIP`), and the diagnostic-only behavior of `test_deployment.py` were all built to keep the suite informative when parts of the world were absent. Components that were deployed (cups, dispenser levers, both Franka arms, gripper command behavior, and the mock end-to-end pipeline) produced actionable `PASS` or `FAIL` signals, while components that were not yet present were marked as `SKIP` rather than counted as broken. The tests that would have required a fully assembled environment, in particular a true parallel two-line run and turntable-mediated cup exchanges across both lines, are therefore blocked on Team 8 deliveries and have been left in the planned-work category. This is consistent with our scope as a quality-control team operating ahead of the full implementation, and it is the main reason our reported coverage stops where it does rather than extending into broader integration scenarios.

### 8.6 End-to-End and Edge Case Limitations

The happy-path end-to-end test is useful because it checks whether task commands can flow to the mocks and whether status messages come back. However, it does not prove that the real scheduler is correct because it bypasses scheduler logic. It also does not prove that the physical robot actions work because the mocks only sleep and publish status.

The edge-case tests are the natural next step. They would evaluate queued orders, parallel assembly lines, robot failures, duplicate commands, and pickup delays. These are the cases most likely to expose system-level

bugs. However, implementing them too early would have required guessing the final behavior of Teams 2, 5, and 6. We therefore left them as planned work until the relevant interfaces stabilize.

# 9 Lessons Learned

**Infrastructure is only valuable if other teams can actually use it.** We could have written more complex tests, but the most useful feature was that each script was easy to run and easy to interpret. A simple `PASS`, `FAIL`, or `SKIP` message helped other teams understand whether an issue was in their code or in the simulation.

**The `SKIP` state was essential.** During a progressive integration project, missing components are normal. Treating missing models as failures would have made the test suite noisy and less trustworthy. Separating `SKIP` from `FAIL` gave a clearer view of what was actually broken.

**Mock robots are useful even when they are simple.** The service and toppings mocks do not perform real robot motion, but they allow the scheduler and communication layers to be tested before the robot controllers are complete. This helps decouple teams during development.

**Interface ambiguities show up quickly when you build against real messages.** Several issues only became obvious once we tried to publish and subscribe using the actual Team 2 message definitions. Writing integration code early forced unclear assumptions into the open.

**Simulation tests should not overclaim.** A passing simulation test means that the simulation behaves well enough for integration testing. It does not guarantee hardware success. This was especially clear for gripper and contact dynamics.

**Small scripts were the right choice.** Standalone files were easier to understand, easier to run, and easier to debug than a more abstract test framework. For a short, multi-team project, that simplicity was more valuable than elegance.

# 10 Conclusion

Team 9 delivered a simulation quality-control suite and mock robot layer for the MEAM 5200 trail mix final project. Our simulation tests check deployment readiness, cup stability, dispenser lever behavior, turntable behavior, gripper command behavior, and arm motion. Our mock robot nodes allow upstream teams to test scheduler and communication behavior without waiting for the real service and toppings robot controllers.

The main value of our work was reducing integration risk. By making simulation failures visible, providing mock robot responses, and surfacing interface gaps early, Team 9 gave other teams a cleaner way to test their own modules. The final edge-case and full pipeline results still depend on stable interfaces from the scheduler and communication teams, but the current codebase provides a clear foundation for those next tests.

## References


[1] MEAM 5200 Course Staff, *Final Project: Trail Mix Bar*, Spring 2026.

[2] Team 2, *trail_mix ROS Package and Topic Interface*, MEAM 5200 Spring 2026.

[3] Team 8, *Gazebo Simulation World*, MEAM 5200 Spring 2026.